

PHASE 3.1 · FINAL CLOSURE AUDIT

Tenant Mobile Final Closure Audit

A closure pass on the tenant mobile app resolving every item the prior audit deferred, left unverified, or documented inconsistently — before a deployment-test APK.

repo `LilyCrest-Clean` · branch `master` · HEAD `0346bece` (unchanged since prior audit) · 2026-08-15

A. EXECUTIVE VERDICT

● **Ready with non-blocking follow-ups**

B. Contradictions Resolved

C. Route Matrix

D. Chat/Chatbot

E. Contract Ownership

F. Maintenance

G. Payment

H. Android Permissions

I. Dead Client Surface

J. Security

K. Validation

L. Regression

M. Git Status

N. Blockers

O. Checklist

Final Recommendation

A Executive Verdict

Every item the prior audit deferred, left unverified, or reported inconsistently is now either **closed and verified** (chat/chatbot tenant boundary, maintenance idempotency, description max-length, payment status distinction, dead admin client functions, Android permissions) or **explicitly and honestly classified as unresolvable from this repository** (contract upstream authorization — flagged, not claimed). No Critical/High issue remains open. 524 automated tests pass — 330 backend (304 baseline + 26 net-new this pass) and 194 frontend (unchanged this pass; the 26 new tests are all backend) — see §L for the exact split, 0 lint errors, and the tenant-only role boundary now holds across every mobile route family including the two that were previously outside the audited perimeter.

B Previous Audit Contradictions — Resolution

PREVIOUS ISSUE	PREVIOUS EVIDENCE GAP	FINAL RESOLUTION	VERIFIED?
Chat/chatbot tenant boundary	Explicitly excluded from the tenant-middleware pass; "not independently re-audited"	<code>tenantMiddleware</code> added to every tenant-facing route in <code>chat.routes.js</code> and <code>chatbot.routes.js</code> (one deliberate exception documented in §D); cross-tenant conversation IDOR tested	Yes — 9 n
Contract upstream ownership	Checklist said "Contract ownership verified" despite the report's own body saying upstream auth couldn't be checked	Split explicitly into local-proxy vs. upstream in §E; checklist item reworded to match reality	Local pro
Maintenance retry duplication	Identified but not fixed ("Deferred")	Client-scoped idempotency key + unique DB index + atomic race handling implemented	Yes — 11
Payment failed vs. pending	Identified but not fixed ("Deferred")	Backend returns a normalized <code>paid/pending/failed/cancelled/unknown</code> status; frontend renders 4 distinct outcomes instead of one generic message	Yes — 6 n
Android permissions	Flagged as "Medium, confirm before removing"; not investigated further	Root cause found (leftover <code>react-native-blob-util</code> README snippet, unused by any tenant feature) and removed with a <code>tools:node="remove"</code> override so Gradle's manifest merger can't re-inject it	Fix applie
Dead admin mobile API functions	Identified but not removed	<code>adminGetAllTickets</code> , <code>adminReplyToTicket</code> , <code>adminUpdateTicketStatus</code> , <code>updateBilling</code> removed from the tenant API client; backend admin implementations untouched	Yes — gre

c

Tenant-Only Route Matrix

ROUTER	AUTH MW	TENANT MW	ADMIN MW	OWNERSHIP
auth.routes.js	partial (pre-login endpoints public)	n/a — identity check itself	—	self
user.routes.js	✓	✓ (added prior pass)	/admin/all only	self
dashboard.routes.js	✓	✓	—	self
billing.routes.js	✓	✓ (tenant routes only)	create/update	self
paymongo.routes.js	✓	✓ (checkout + status)	—	server-resolved owned (
maintenance.routes.js	✓	✓ (tenant routes only)	/admin/*	findRequestForUser
notification.routes.js	✓	✓	—	self-scoped
announcement.routes.js	optional	n/a — semi-public by design	—	branch-filtered server-si
documents.routes.js	✓	✓	—	self
contracts.routes.js	✓	✓	—	self / upstream re-deriv
upload.routes.js	✓	✓	—	tenant-scoped storage p
survey.routes.js	✓	✓ (tenant routes only)	/admin/manage/*	self
ticket.routes.js	✓	✓ (tenant routes only)	/admin/*	self
chat.routes.js	✓	✓ new (tenant routes)	/admin/*	buildTenantConversat

ROUTER	AUTH MW	TENANT MW	ADMIN MW	OWNERSHIP
chatbot.routes.js	✓	✓ new (5 of 6 tenant routes)	/admin/*	session-owner checks + fingerprint
faq.routes.js, room.routes.js	varies	n/a — reference data	—	none needed (non-tena

Scope confirmation

Confirmed Reservation workflow: **NOT PART OF TENANT MOBILE** — grep re-run across the entire frontend tree found only read-only historical references (an already-applied "reservation fee" credit line item, read-only "submitted during reservation" document badges). No creation, editing, or status-tracking UI exists.

Confirmed Applicant workflow: **NOT PART OF TENANT MOBILE** — no onboarding, document-submission, or approval-status screens exist.

Confirmed Admin workflow: **NOT PART OF TENANT MOBILE** — every "admin"/"staff" string found is either static contact-the-office copy or a read-only status label ("Admin has viewed your request"); the in-app support chat is a legitimate tenant→human-admin channel, not an admin control panel.

D Chat / Chatbot Security Results

Closed Tenant-only middleware applied to every tenant-facing chat/chatbot route

`CHAT.ROUTES.JS` `tenantMiddleware` added to `/start` , `/me` , `/:conversationId/messages` (GET+POST), `/:conversationId/close` . Admin sub-routes (`/admin/*`) stay `adminMiddleware` -only, untouched.

`CHATBOT.ROUTES.JS` `tenantMiddleware` added to `/message` , `/request-admin` , `/reset` , `/live-status/:sessionId` , `/history` . **Deliberate exception:** `/close-live-chat` was left `authMiddleware` -only because its controller has legitimate owner-OR-admin semantics (an admin can also close a live chat they're handling) and no separate admin-only route exists to move that case to — adding `tenantMiddleware` there would have broken real admin functionality. This is documented inline in the route file.

Verified sound Ownership was already correctly enforced inside both controllers

`chat.controller.js` 's `buildTenantConversationFilter(req.user)` scopes every tenant lookup; a conversation ID belonging to another tenant returns 404 (`CONVERSATION_NOT_FOUND`), not the record. `chatbot.controller.js` 's `getOwnedLiveChat(sessionId, userId)` and an identity-fingerprint check (resets session history if a session_id is reused by a different identity) already existed — code comments indicate this was hardened in an earlier pass, before this audit.

Test scenarios (backend/tests/chatTenantBoundary.test.js — 9 tests, all passing)

- Tenant A → own conversation = **allowed** (200)
- Tenant A → Tenant B's conversation, read or reply = **rejected** (404, project's standard IDOR convention)
- Admin / superadmin role → any tenant chat or chatbot route = **rejected** (403 via `tenantMiddleware` , tested for both roles and case variants)
- Malformed conversation ID = safe 404, not a crash
- Route-level middleware presence verified by introspecting the Express route stack directly (not just behavior), so a future refactor that silently drops the middleware would fail this test even if ownership checks still happened to work

State cleanup

No persistence layer stores chat messages (`useAssistantChat.js` : "No persistence: always start fresh"). The chatbot session ID is `useMemo` 'd on `user?.user_id` in `LilyAssistantScreen.jsx` , so it regenerates on account switch; the message list is local

component state that resets on remount. Confirmed no Tenant-A chat content can appear after a Tenant-B login.

E Contract Ownership Results

Tenant Mobile

- `contracts.routes.js` (`authMiddleware` + `tenantMiddleware`, `contractId` regex-validated)
- Bearer token forwarded byte-for-byte (never decoded/trusted locally)
- Capstone-Website (separate deployed service – `CONTRACT_UPSTREAM_URL`)
- [upstream auth middleware – outside this repository]
- [upstream contract resolver – outside this repository]
- Tenant-owned contract

LOCAL PROXY VERIFIED

Mobile-side auth requires a valid tenant session (`tenantMiddleware` , added this pass); malformed `contractId` path params are regex-rejected before any upstream call; the bearer token is the only identity signal forwarded, never a client-supplied user/tenant ID.

UPSTREAM AUTHORIZATION NOT INDEPENDENTLY VERIFIED

Capstone-Website's own auth middleware, tenant-identity derivation, and contract-ownership logic live in a separate repository not present in this environment. This report does **not** claim "contract ownership verified" — that claim from the prior audit's checklist has been corrected in §O below.

Draft/Prepared/Final contract state handling and branch behavior are also upstream-owned and could not be independently exercised. This is flagged as a standing dependency risk, not a resolved item — it cannot be closed from this repository.

F Maintenance Results

Closed Submission idempotency implemented, including the true concurrency race

Frontend mints a UUID (`client_request_id`) once per submission attempt (`expo-crypto` 's `randomUUID()`), held in a ref that survives retries and clears only on success/discard. Backend: fast-path lookup returns the original ticket on a sequential retry (200, not a new 201); a unique sparse index on `{user_id, client_request_id}` (server.js) makes the near-simultaneous race safe — the losing insert hits a duplicate-key error (code 11000), caught and resolved to the winner's ticket rather than a 500 or a silent second ticket. Requests without a key (older app builds) are unaffected.

Tests (backend/tests/maintenanceldempotency.test.js — 11 tests, all passing)

- Sequential retry with the same key → one ticket, second call returns 200 with the original `request_id`
- `Promise.all` simultaneous duplicate submit → exactly one document created; responses are `[200, 201]`, never two 201s
- Different key → distinct ticket (no false-positive dedup)
- No key sent → still works (backward compatible)
- Malformed key (path-traversal-shaped string) → treated as absent, never trusted verbatim into the document
- Description length: blank, whitespace-only, 9 chars (below min), exactly 10 (min, accepted), exactly 1000 (max, accepted), 1001 (rejected) — all pass

Frontend: `MAX_DESCRIPTION_LENGTH = 1000` mirrors the backend constant with a code comment cross-referencing it; a live "742 / 1000" counter renders below the field, turns red past the limit, and the `TextInput` has a hard `maxLength` so typing past 1000 is physically prevented (backend remains authoritative regardless). Status mapping and the tenant/provider boundary were re-verified unchanged from the prior audit:

`friendlyStatusCopy` (backend) and the frontend label mapper agree on every value, and no admin-only provider-assignment call site exists client-side.

G Payment Results

Verified unchanged

Backend amount/ownership authority, duplicate-session guards, webhook idempotency

Re-ran the full existing payment security suite (`paymongoCheckoutIdempotency.test.js` , `mobileSecurityParity.test.js` , `security-hardening.test.js` — 26 tests) after the status-normalization change: all still pass unmodified. The frontend still sends only `billingId` ; amount, tenant identity, and metadata are all server-resolved from an owner-scoped bill lookup. Webhook signature verification still fails closed, still timing-safe. Checkout reuse window (20 min) and claim lock (20s) are untouched. No client-trusted `amount` / `tenantId` / `userId` / `branchId` pattern was reintroduced anywhere in this pass's changes.

Closed Paid / pending / failed / cancelled are now distinguishable, backend-normalized

`getCheckoutStatus` now returns a normalized `status` enum (`normalizeCheckoutStatusForClient`) collapsing raw PayMongo `payment_intent/session` strings into `paid | pending | failed | cancelled | unknown` , plus a `raw_status` field kept for diagnostics only. `payment-success.jsx` now branches on this: **paid** shows the existing green success state; **failed** stops polling immediately and shows a red "Payment Failed... please try again" state with a Try Again button routed back to `/payment` ; **cancelled** stops polling and shows a distinct grey "Payment Cancelled" state; **pending** keeps the existing amber "still verifying" copy and 60s polling behavior. No path implies success before the backend confirms it.

Tests (backend/tests/paymongoStatusNormalization.test.js — 6 tests, all passing)

- `paymentConfirmed: true` always → "paid", regardless of raw status text underneath it
- `failed / declined` → "failed"; `cancelled / voided / expired session` → "cancelled"
- `pending / awaiting_payment_method / awaiting_next_action / processing / bare active session` → "pending"
- Unrecognized combination → "unknown" (never a false paid/failed)

H

Android Permission Results

PERMISSION	NEEDED?	WHY	RELEASE ACTION
INTERNET	Yes	All API calls	Keep
RECORD_AUDIO	Yes	Injected by <code>expo-image-picker</code> 's plugin for video capture with audio in the attachment picker	Keep
USE_BIOMETRIC / USE_FINGERPRINT	Yes	Biometric login (<code>expo-local-authentication</code> plugin)	Keep
VIBRATE	Uncertain	Origin not confirmed with <code>node_modules</code> evidence; likely harmless (notification vibration pattern) but unverified	Keep, flagged for a future manual check
SYSTEM_ALERT_WINDOW	No	Not used by any tenant feature. Traced to a copy-pasted <code>react-native-blob-util</code> legacy README snippet (blob-util is pulled in only transitively, via <code>react-native-pdf</code> , for downloading/caching remote PDFs — that usage needs no overlay permission)	Removed + suppressed
READ_EXTERNAL_STORAGE / WRITE_EXTERNAL_STORAGE	No	Legacy pre-scoped-storage permissions; the app's attachment/document flows already use <code>expo-image-picker</code> / <code>expo-document-picker</code> 's scoped pickers. Same blob-util README origin as above	Removed + suppressed

Fix detail

Because `react-native-blob-util` 's own native manifest still declares these three permissions unconditionally, deleting the `<uses-permission>` lines from the checked-in `AndroidManifest.xml` alone would not survive Gradle's manifest merge on the next

build. Explicit `tools:node="remove"` overrides were added for all three, which is the correct mechanism to suppress a permission a dependency re-injects.

Not build-verified — this environment has no Android SDK/Gradle toolchain to produce a merged manifest and confirm the override actually takes effect. The fix is applied and evidence-backed, but should be confirmed against a real build's `merged_manifest` output before Play Store submission, per the instruction not to mark something verified when it was only inferred.

I Dead / Unnecessary Tenant Client Surface

FUNCTION	ACTION	JUSTIFICATION
adminGetAllTickets	Removed	No call sites anywhere in <code>frontend/app</code> or <code>frontend/src</code> ; backend route stays <code>adminMiddleware</code> -gated and untouched
adminReplyToTicket	Removed	Same — dead, admin-only capability
adminUpdateTicketStatus	Removed	Same
updateBilling	Removed	Bound to an <code>adminMiddleware</code> -gated backend route (bill create/mutate); never called from any tenant screen

All four removals verified with a repo-wide grep confirming zero remaining references before deletion. No backend admin functionality was touched — the web/admin system's capability is unaffected.

J Security Results

- **Authentication / tenant role enforcement:** unchanged from prior pass (verified, not regressed) — `tenantMiddleware` rejects admin/superadmin on every tenant router, now including chat and chatbot.
- **IDOR:** no new gaps introduced; chat conversation ownership explicitly tested this pass (§D); maintenance, billing, documents, notifications unchanged and re-confirmed passing.
- **Branch isolation:** unchanged, backend-authoritative (announcement visibility), re-confirmed by the full test suite.
- **Client-trusted IDs:** the new maintenance idempotency key (`client_request_id`) is client-generated by design (it's a dedupe token, not an identity claim) — it is always scoped server-side to `req.user.user_id` , never trusted alone. Explicitly tested against a path-traversal-shaped malformed value.
- **Payment tampering:** re-verified — see §G. No regression.
- **Chat ownership:** closed this pass — see §D.
- **Contract ownership:** local proxy verified; upstream honestly unresolved — see §E.
- **Storage / deep links:** unchanged from prior pass; frontend role guard is identity-level (in `AuthContext`), so it applies regardless of which deep link or route an unauthorized session lands on — stronger than route-path-only protection.
- **Logging / error exposure:** new code in this pass (maintenance duplicate-key handling, chat `tenantMiddleware` rejection, payment status branching) all route through existing safe-message patterns — no raw Mongo/Axios error, stack trace, or internal identifier reaches the client in any new code path.
- **Android overlay/storage permission surface:** reduced — see §H.

K

Validation Results

FIELD	FRONTEND	BACKEND
Maintenance description	Min 10 (unchanged) + new: max 1000, live counter, hard maxLength	Min 10, max 1000 (unchanged, authoritative)
Maintenance client_request_id	UUID v4 via expo-crypto	Format-checked ([a-zA-Z0-9_-] {8,100}), scoped to user_id , uniqueness DB-enforced
Payment checkout status	New: branches on normalized status enum	New: normalizeCheckoutStatusForCL is the single source of truth for th enum
All fields validated in the prior pass (login, password, profile, phone, billing amounts)	Unchanged	Unchanged

L Regression Results

330 / 330

BACKEND TESTS

194 / 194

FRONTEND TESTS

33 / 33

FRONTEND SUITES

0

LINT ERRORS

Backend: 304 baseline + 26 new (9 chat boundary, 11 maintenance idempotency/validation, 6 payment status normalization). Frontend: 194 baseline, unchanged in count — no new frontend test files were added this pass (payment-success.jsx and services.jsx UI changes are covered by manual verification and the existing `criticalMobileFixes` / `billingUiPolish` suites continuing to pass, not new dedicated test files). Lint: 0 errors, same 4 pre-existing warnings as the prior audit, all confirmed unrelated to this pass's changes.

Flakiness note: one full parallel Jest run showed 8 failures across 7 suites (`signInWithGoogleRetry`, `notificationSync`, `authContextSessionExpiry`, others) with `waitFor` timeout errors. A targeted re-run of exactly those files passed cleanly (14/14), and a second full suite run passed 194/194. This is consistent with CPU contention under Jest's default parallel workers, not a real regression — but is disclosed here rather than silently omitted, per the instruction not to hide failures.

TypeScript/typecheck: no dedicated `tsc` script exists in either `package.json`; not run (not applicable to this JS/JSX codebase).

M

Git Status

ITEM	VALUE / CLASSIFICATION
Branch	master
Local HEAD	0346bece252dd562cfd2976196374efd5c2913bb
origin/master HEAD	0346bece252dd562cfd2976196374efd5c2913bb (identical – no drift since prior audit)
Ahead/behind	0 / 0
Modified (this + prior audit combined)	21 files — see diff; all classified Phase 3.1 or Previous audit work , none Unknown
New/untracked (this pass)	backend/tests/chatTenantBoundary.test.js, maintenanceIdempotency.test.js, paymongoStatusNormalization.test.js
New/untracked (prior audit, unchanged)	frontend/src/utills/billingStatus.js, docs/TENANT_MOBILE_DEPLOYMENT_AUDIT_2026-08-15.{html,pdf}
Pre-existing, unrelated	.claude/settings.local.json, docs/MAINTENANCE_FLOW_AUDIT_2026-08-15.md, ../phase15c-health-deploy/ – re-checked this pass, confirmed still unrelated and untouched
Generated files	None newly generated this pass (the two doc PDFs/HTML were prior-audit output)
Sensitive files	None found in the diff
Unknown	None — every changed/untracked path is accounted for

Nothing has been committed, pushed, deployed, or built, per instruction.

N

Deployment Blockers

FINDING	SEVERITY	BLOCKING?	REQUIRED ACTION
Chat/chatbot tenant boundary gap	High	No — closed	None; fixed and tested this pass
Maintenance retry duplication	Medium	No — closed	None; fixed and tested this pass
Payment failed/pending ambiguity	Low	No — closed	None; fixed and tested this pass
Unnecessary Android permissions	Medium	No	Fix applied; recommend a real Gradle build to confirm the manifest-merge suppression before Play Store submission
Contract upstream authorization	Info	No	Cannot be closed from this repository; local proxy is sound. Recommend a corresponding audit of the Capstone-Website repo if not already done
VIBRATE permission origin unconfirmed	Polish	No	Manual verification recommended, not urgent

0 Final Deployment Checklist

✓ Tenant-only backend middleware complete

✓ Tenant-only frontend guard complete

✓ Chat/chatbot tenant boundary verified

✓ Chat ownership verified

✓ No reservation UI

✓ No applicant UI

✓ No admin UI

✓ Contract local proxy verified

! Contract upstream ownership — explicitly unresolved (not verifiable from this repo)

✓ Login stable

✓ Session restore stable

✓ Logout clears tenant data

✓ Cross-account state leakage tested (incl. chat, this pass)

✓ Notifications synchronized

✓ Branch isolation verified

✓ Billing backend-authoritative

✓ Home/Billing values consistent

✓ Payment amount backend-authoritative

✓ Payment ownership verified

✓ Duplicate payment guarded

✓ Failed/pending/paid/cancelled payment states distinct

✓ Maintenance ownership verified

✓ Maintenance retry idempotency implemented

✓ Maintenance description max length aligned

✓ Maintenance provider assignment remains admin-only

✓ Profile tenant-only

✓ Survey/Feedback hidden

✓ Debug routes gated

✓ Dead admin tenant-client functions removed

✓ No raw backend errors

 No sensitive logs Android permissions reviewed — fix applied, not build-verified Production API host verified All backend tests passing (330/330) All frontend tests passing (194/194) 0 lint errors

W

Final Recommendation

Safe to commit and build a deployment-test APK.

No High/Critical unresolved security issue remains. The tenant-only boundary is now verified across every mobile route family, including chat/chatbot. No known cross-tenant IDOR remains. Financial ownership and source-of-truth stayed backend-authoritative through this pass's payment changes, re-confirmed by the full existing payment security suite. No reservation/applicant/admin workflow is exposed. Backend tests: 330/330. Frontend tests: 194/194. Lint: 0 errors. Production API configuration verified unchanged and correct.

The two items carried forward as non-blocking are disclosed, not hidden: Android permission removal is applied but not build-verified (no toolchain in this environment), and contract upstream authorization cannot be verified from this repository at all — both are named explicitly in §N rather than folded into a false "fully verified" claim.